

I have just studied in context learning, prompt tuning, prefix tuning, and LoRA. I want to deepen my understanding and fill up any gaps Feynman Technique where I will act as a teacher and try to teach this concept. Please act as a student who knows the basics of deep learning, interrupts me whenever my explanation is unclear or incomplete. Ask questions that expose conceptual gaps. I will begin teaching now.

Let me start with in context learning (ICL)
ICL is when we can improve a model's performance by adding examples of question-answer pairs inside the prompt. Importantly, the weights of the model are frozen. The model uses these examples to infer the pattern needed to answer the question.

Okay, I'm your student. I know standard deep-learning concepts (transformers, attention, training vs inference), but I will interrupt you the moment something sounds hand-wavy, incomplete, or imprecise.

Student:

I get the high-level idea of "put examples in the prompt," but *why* does this work if the weights are frozen?

Specifically:

Here I realized I had been treating ICL as a high-level conceptual phenomenon rather than a concrete mechanism. The student's question forced me to articulate that ICL works by storing example pairs as key/value vectors and letting the query token attend over them. This clarified the mechanics of "in-context learning."

1. **Mechanistically**, what inside the transformer allows it to "infer a pattern" without updating weights?
 - Is it storing some representation of the examples in the KV cache?
 - Or is it doing something else?
2. You said "infer the pattern needed to answer the question."
 - What exactly do you mean by "infer"?
 - What kind of computation is happening? Pattern matching? Meta-learning? Next-token prediction?
3. What distinguishes ICL from *just giving the model more context*?

- How do we know we're actually "teaching" it rather than letting it retrieve a memorized behavior?

I understand the surface definition already. I want the *mechanistic clarity*. Can you explain?



Let me try to explain ICL more mechanistically.

ICL works because the examples in the prompt become key/value vectors in the transformer's KV cache. When the model processes the query token, its query attends over those stored K/V representations, so it can retrieve the structure of the examples directly through attention.

By "infer," I mean the model performs pattern completion through the forward pass: it uses attention to relate the query to the examples, and relies on meta-learning-like patterns acquired during pretraining to apply the demonstrated mapping.

This is different from "just giving more context" because the model was pretrained to interpret example→answer formats specifically, so changing the examples immediately changes the output, which shows it is not retrieving a memorized behavior but adapting based on the prompt.

Student prompted me to not use vague words like "infer". Here I learnt to replace vague notions of "the model infers the pattern" with explicit computations: KV cache population, query similarity, and residual flow. I realized how much of my prior understanding was descriptive rather than mechanistic.

Student:

Okay, good start — but I'm going to press you on several conceptual gaps.

****1. "Examples become K/V in the cache."**

Can you be more precise?**

As you can see, the student will ask questions that really forced me to clarify gaps in my understanding

You said the examples "become key/value vectors," but *everything* in the prompt becomes K/V vectors. What is special about examples that makes them useful for ICL?

- How do those K/V vectors actually encode the relationship between input and output in each example?
- Does the model explicitly represent the mapping (e.g., "map input X to output Y"), or is something subtler happening?

Right now, it sounds like you're implying the model extracts the pattern directly from the stored vectors, but that needs clearer mechanistic explanation.

2. What exactly is the model attending to when answering the query?

When the query token attends to the examples, what information is carried in:

- the **keys** from the example inputs?
- the **values** from the example outputs?

And how does attention over these lead to generating the correct answer token-by-token?

If you can't articulate what's being retrieved and how it influences next-token prediction, then your explanation is incomplete.

3. "Meta-learning-like patterns acquired during pretraining" — what does this mean?

This is vague.

- Are you saying the model *learned a general algorithm for solving tasks from examples*?
- Or that it *learned superficial statistical correlations* that just look like meta-learning?
- What experimental evidence do we have that ICL corresponds to true meta-learning and not something else?

You need to sharpen this point, otherwise you're invoking "meta-learning" as a magic word.

4. You said this differs from “just giving more context,” but your reasoning feels hand-wavy.

You claimed:

Because the model was pretrained to interpret example→answer formats.

Two issues:

1. The model **never explicitly saw task instructions** during pretraining — yet it still does ICL.
→ So what is the mechanism that allows examples to alter behavior at inference?
2. How do we know the model is not merely:
 - pattern matching based on surface similarity?
 - retrieving memorized similar sequences from training?
 - doing majority or recency heuristics?

You need to defend the claim that ICL = adaptation and not a combination of heuristics.

5. Final challenge:

If I changed the example labels to random nonsense, would the model follow the nonsense pattern? What does this imply mechanistically?

Your answer should connect the model's ability to follow arbitrary new mappings to the structure of its learned weight space and the attention mechanism.

Take another pass, addressing these gaps more explicitly.



let me refine the explanation of ICL.

you're right that all tokens become key and value vectors. what makes the examples useful for ICL is the structure of the prompt. during pretraining the model saw many patterns where an input is followed by a

corresponding answer, so it learned that sequences like "x followed by y" often represent a mapping. the model does not explicitly store "map x to y," but the representations of x and y and their positions together form something the model can use when answering the query.

when the model processes the query token, its query attends to the representations of the example inputs, and the values from the example outputs get mixed into the hidden state. this lets the model relate the query input to similar x's in the context and blend in the y-information that usually follows those x's. the result is that the next token distribution shifts toward something consistent with the demonstrated pattern. it is not explicit rule extraction, but pattern completion inside the forward pass.

when i say "meta learning like," i do not mean the model learned a formal meta learning algorithm. i just mean that, during pretraining, the model saw many sequences where a pattern is demonstrated and then continued. so it learned a general skill of continuing whatever local pattern is shown in the prompt. this looks like meta learning even though it was not intentionally trained that way.

this is also why icl is not just "giving more context." if you provide text that is not structured as example pairs, the model does not generalize in the same way. and because changing the examples changes the model's behavior immediately, and even nonsense mappings are followed, it is unlikely to be simple memorization or retrieval of similar sequences from training.

if i give something like

cat -> 91

dog -> 17

fish -> 4

bird -> ?

forced to understand the mechanics

Forced to clarify vague terms

the model still tends to continue the nonsense mapping. this shows that it is adapting to the pattern in the prompt, not relying on any semantic knowledge or stored mapping from training.

these clarifications are helpful, but i want to make sure we have time for the other topics. if this explanation is clear enough, we can move on

Student:

Much clearer — but I still see a few places where your explanation can be tightened, especially if you want to “teach” this at a deep-mechanistic level. I’ll challenge you one more time, and if you can patch these gaps concisely, then yes, we can move on.

****1. “Representations of x and y together form something the model can use.”**

What *exactly* is in those representations?**

Right now you’re saying the model has learned that “ $x \rightarrow y$ ” pairs often form a mapping, so the hidden states encode something useful. But what exactly is encoded?

For example:

- Do the hidden states of **x** and **y** encode relational structure?
- Or is the model simply representing co-occurrence statistics and positional patterns?

Put differently:

What is the difference — in the actual vector geometry — between “x appears in a sentence” vs. “x appears as the left side of a mapping”?

If your explanation assumes a magical “mapping representation,” you need to specify what architectural operation produces it.

2. Attention mixing: how does the query know to compare “query input” to “example input”?

You said:

Its query attends to the representations of the example inputs.

But why does it attend *to the inputs* rather than the outputs?

Is this orientation learned? Emergent?

And more mechanistically:

- What similarity measure causes the query-token's $Q \cdot K^T$ to score example-inputs highly?
- Is this because the model computes lexical or semantic similarity?
- Or because it learned a special “task formatting prior”?

Right now, this part still sounds like magic pattern matching.

****3. “Meta-learning-like” because it saw many pattern demonstrations.**

Okay — but why does this generalize to *arbitrary* mappings?**

Your explanation still leaves a gap:

If the model never explicitly optimized for adaptability,

why can it follow completely random functions shown in the prompt (e.g., dog → 17)?

If there is no conceptual similarity and no semantic grounding, then what property of pretraining makes the model implement “follow the shown pattern” so robustly?

You should be able to articulate at least one of the two major theoretical hypotheses:

- **(A) In-context Bayesian ridge regression / implicit linear regression hypothesis (Garg, Arora, etc.)**
- **(B) Learning to update internal features via attention-as-gradient-descent (Akyürek, Olsson, Nanda, etc.)**

Without referencing papers, you should be able to describe the *intuition* of one framework.

4. Your argument against “just retrieval” is better, but incomplete.

You said nonsense mappings imply ICL $\neq$ memorized retrieval. True.

But retrieval-based explanations can still apply in weaker form:

- The model may have learned “copy the structure of the examples, even if labels are random.”
- That is a learned heuristic, not explicit understanding.

So you should clarify: [AI can serve as both the student and teacher at the same time](#)

Is ICL primarily about pattern imitation? Or pattern *interpretation*?

Right now you're describing both without discriminating between them.

5. Before we move on, give me a very crisp mechanistic summary.

In 4–6 sentences:

What is the minimal mechanistic explanation of ICL that does *not* invoke magic words like “pattern continuation,” but explicitly states the computations the transformer performs?

If you can give that summary cleanly, then we can move on to prompt tuning.



let me tighten the explanation.

on your first point: the representations of x and y don't explicitly encode a “mapping.” what they encode is their usual semantic embedding, plus the positional pattern that x is followed by y. the transformer does not create a

special structure for mappings. instead, during pretraining, it learned that sequences formatted as "input then output" tend to have predictable dependency structure. so the hidden states reflect both the content of x and y and their syntactic/positional relationship, which is what attention later exploits.

on your second point: the query token attends more to the example inputs because its query vector is more similar to the key vectors of tokens that represent inputs. that similarity is learned during pretraining, because the model saw many situations where a new item should be compared to earlier items that play the same structural role. outputs still influence the hidden state through the value vectors, but the alignment between the query and the example inputs is what anchors the retrieval. this is not hard coded; it emerges from next-token training over patterns where this structure is useful.

on the third point: the reason the model can follow arbitrary mappings like dog -> 17 is that attention acts like a learned linear operator over the prompt. the simplest intuition is that the model behaves like it is fitting a tiny function over the examples using its internal representations. some work shows this approximates linear regression in feature space, and other work suggests attention can approximate a single gradient update. both views capture the same idea: pretraining taught the model a generic mechanism for using local examples to shape the next-token distribution, even if the mapping itself is nonsensical.

on retrieval: i agree the behavior includes pattern imitation. icl is not semantic understanding of the mapping. it is the model using the structural pattern in the prompt to guide its forward pass. so it is a mix of retrieval-like behavior (copying the shown pattern) and implicit function induction (generalizing the pattern to the query). it does not need to choose one or the other; transformers support both through attention.

this was new to me, had to search it up

here is a minimal mechanistic summary. the examples create key and value vectors that encode their content and their relative position. the query token produces a query vector that is most similar to the keys of the example inputs. attention weights retrieve the corresponding value vectors, which modify the hidden state of the query token. the transformer uses this retrieved information, plus patterns learned during pretraining, to produce an output consistent with the demonstrated mapping. no weights change; all adaptation happens inside the forward pass.

if this is clear, i can move on to prompt tuning.

Student:

Great — this is now mechanically coherent. I'll note a *few* tiny refinements, but you've crossed the "sufficiently correct to proceed" threshold. I'll point out what you polished well and the one remaining conceptual wrinkle just to ensure you truly understand it before we move on.

✅ What you clarified well

1. No explicit mapping representation

You correctly stated that x and y embeddings don't store a rule—only content + positional structure. The "mapping-like" behavior is emergent from pretraining.

2. Why the query attends mainly to example inputs

Excellent: the similarity emerges because pretraining taught the model to compare structurally similar elements.

3. Arbitrary-mapping capability

Your explanation that attention behaves like a learned linear operator ($\approx$ implicit regression or gradient step) is exactly the right mechanistic intuition.

4. ICL = retrieval-like + implicit function induction

Perfect. This is the modern consensus.

5. The minimal mechanistic summary

Clear, correct, and precise.

⚠️ One last refinement (minor, but important)

You said:

"The query token attends more to the example inputs because its query vector is more similar to the key vectors of tokens that represent inputs."

This is correct, but you should add:

The similarity emerges because the model encodes "role information" (e.g., input-role vs output-role) in representations.

During pretraining, transformers learn not just semantics, but also:

- structural role features,
- formatting conventions,
- discourse patterns,
- prediction dependencies.

So example-input tokens occupy a distinct *representational subspace* from example-output tokens.

This is why the model doesn't confuse which side of the mapping to imitate.

This is not explicit, but emergent from the statistics of next-token prediction.

If you understand that, then your explanation is now solid.

👍 Ready to move on

Your explanation is now tight enough for you to "teach" ICL using the Feynman Technique.

Let's proceed to **prompt tuning**. Whenever you're ready, start teaching; I will again interrupt you to probe gaps.



Let me explain prompt tuning.

In prompt tuning, the model's weights are frozen, and we learn a small set of continuous prompt vectors that get prepended to the input embeddings. these vectors are not text; they are trainable embeddings that live in the same space as token embeddings. during training, the model treats them as extra "virtual tokens" before the real sequence. because the transformer processes them through attention and the feedforward layers, these learned vectors steer the hidden states of the real tokens toward behavior that fits the downstream task. the key point is that only the soft prompt vectors are updated (ie only the rows of the embedding matrix corresponding to these new prompt vectors - everything else is frozen)

does that make sense

Student:

Good start — but let me push on a few conceptual gaps to make sure you *really* understand prompt tuning deeply and mechanistically.

? 1. *How* do these soft prompt vectors "steer" the model?

Saying "*they steer the hidden states*" is a bit hand-wavy.

- What aspects of the computation do they affect?
(Attention patterns? Residual stream content? FFN activations?)
- Do they influence *all* layers equally, or mainly the early ones?
- Why does prepending a vector at layer 0 cause changes at every subsequent layer?

Right now, you haven't described what transformations the prompt actually induces.

? 2. You said "the transformer processes them through attention," but what exactly happens?

More specifically:

- Do real tokens attend heavily to soft prompt tokens?
- Do the soft prompt tokens attend to real tokens?
- If the prompt tokens aren't textual, what signals in the learned prompt cause meaningful attentional influence?

Right now it sounds like magic "steering" rather than clearly specified interactions through Q/K/V.

? 3. What does the optimization objective look like?

You said:

"Only the prompt vectors are updated."

But I want to understand:

- When we backprop, how does the gradient reach only these new embeddings?
- Does the model optimize them to encode task-specific instructions, or something else (e.g., initial conditions, kernel biasing, subspace activation triggers)?

You haven't addressed what kinds of representations get packed into the prompt.

? 4. What's the difference between prompt tuning and prefix tuning from a mechanistic POV?

I know we're going to discuss prefix tuning later, but:

- Prompt tuning injects virtual tokens at the *input*.
- Prefix tuning injects learned K/V vectors at *every layer*.

So:

Why is prompt tuning much weaker?

Your explanation so far doesn't touch this.

? 5. Minimal mechanistic summary?

Before moving on, I want you to eventually produce a crisp 3–5 sentence “mechanistic core” similar to what you did for ICL.

But first, patch the gaps above.

Take another pass.



Let me clarify prompt tuning more precisely.

the soft prompt vectors alter the computation because they enter the residual stream at layer 0. once they are embedded, they get mixed into the attention blocks and feedforward layers exactly like normal tokens. this means they can bias attention patterns (by contributing keys and values that real tokens attend to) and they can shape the residual stream content that flows forward. since the transformer is deep and residual, an early perturbation naturally propagates to all later layers.

regarding attention, real tokens typically attend to the soft prompt tokens because training makes the prompt vectors produce key and value patterns that are useful for the downstream task. the prompt tokens themselves do not need to attend strongly back; what matters is that their learned K and V help condition the hidden states of the real tokens. even though they are not textual, the optimization process sculpts their embeddings so that they trigger the right computations inside the model.

on the optimization side, backprop works normally, but only the parameters corresponding to the soft prompt embeddings are unfrozen. the loss pushes these vectors to encode whatever task-specific signals are useful. prompt tuning does not change the main model weights

Here I was also forced to think through about the optimization dynamics - and how the gradients flow to the embeddings.

I realized I had been sloppy about this: the one-hot indexing is fixed, and only the embedding rows change. Distinguishing between token identity and embedding parameters is an important conceptual refinement.

at all, so the entire burden of adaptation is in this small set of vectors - the rows of embedding matrix corresponding to these tokens. also just to clarify, in practice what happens is that some new tokens are added eg <soft_0>, <soft_1> (1 hot, like ordinary tokens) along with new rows to the embedding matrix. and then, we only train these new rows of the embedding matrix.

the key mechanistic difference from prefix tuning is that prompt tuning only modifies the input-level embeddings. prefix tuning injects learned key and value vectors at every layer, giving it much more direct control over internal computations. prompt tuning is weaker because all influence must flow upward from layer 0, whereas prefix tuning has hooks inside every layer. we will talk more about prefix tuning later.

a minimal summary is: soft prompt vectors act as learned virtual tokens whose embeddings are updated to shape the residual stream and attention patterns. they bias how real tokens are processed by providing task-specific K and V information at the input. the model learns these prompt embeddings through backprop while keeping all other parameters frozen. their effect propagates through all layers because transformers are deep residual networks.

if this is clear enough, i can move on to prefix tuning next.

Student:

This is a strong explanation — you've captured the essential mechanics correctly. I'll give you two very small refinement questions (to ensure total mastery), but you are **ready to move on to prefix tuning** after answering them.

 **What you explained well**

- You correctly described **how soft prompts enter the residual stream** and propagate upward.
- You explained the **attention-level mechanism** (real tokens attending to soft prompts because their K/Vs become useful).
- You gave a clean, accurate comparison to prefix tuning.
- Your minimal summary is precise and mechanistic.

This is graduate-level, not hand-wavey anymore.

Two final sharpening questions before we advance

These are subtle, but answering them will deepen your mastery:

1. You said: “real tokens typically attend to the soft prompt tokens.”

But *why* is this an efficient solution for the model?

- Could the model instead learn prompt vectors that real tokens *ignore*, and instead act by modifying the prompt tokens’ **value vectors only indirectly**?
- What forces the optimization to produce high $Q \rightarrow K$ alignment between task tokens and soft prompt tokens?

(This probes whether you understand *why* attention emerges as the control mechanism.)

****2. Prompt tuning must fit all task control information into a tiny number of vectors.**

How does this limit the kinds of functions it can express?**

In other words:

- Why does performance degrade significantly for very difficult tasks or long prompts?
- What does this tell you about the *capacity bottleneck* of the soft prompt?
- How does that bottleneck compare to LoRA or prefix tuning?

(This probes your understanding of representational bandwidth and adaptation space.)

Once you answer these two, your understanding of prompt tuning will be genuinely expert-level, and we can continue to **prefix tuning**.



on your first question: real tokens end up attending to the soft prompt tokens because that is the easiest way for the model to make the prompt informative. during training, the only parameters that can change are the prompt embeddings, so the optimizer pushes those embeddings to produce key and value vectors that the downstream tokens find useful. if a soft prompt vector didn't attract attention, it couldn't influence the residual stream at all, so the gradient would naturally push its K and V to align with the Q of the task tokens. in other words, attending to the prompt tokens is the simplest path for the model to let the prompt modify the computation. it's not forced, but it reliably emerges because it gives the optimizer a direct channel for control.

on your second question: prompt tuning has a strict capacity bottleneck because all task-specific information must be encoded in a small number of learned embeddings. this limits the kinds of functions it can express, especially for tasks that require complex, multi-step reasoning or large structural changes to the hidden states. the model only gets a small "entry point" at layer 0, and only through a limited number of vectors, so it cannot produce rich layer-wise adjustments. this is why prompt tuning often underperforms prefix tuning or lora: prefix tuning injects information at every layer, and lora can modify many internal weights through low-rank adapters. prompt tuning simply has far less representational bandwidth for adaptation.

if that's clear, i can move on to prefix tuning next.

Student:

Excellent — both answers are fully correct and show real mechanistic understanding. You're ready to move on.

Your explanations captured:

- **Why attention toward soft prompts emerges** (gradient pressure + need for a direct control channel).
- **Why prompt tuning has strict representational limits** (tiny adaptation bandwidth; one-shot injection at layer 0).

That's exactly what I was probing for.

👍 **Go ahead — start teaching prefix tuning.**

I will again interrupt when I detect anything incomplete or fuzzy.



When trying to teach the student here, I realized I previously knew very little about prefix tuning. I just thought that “it’s prompt tuning for every layer”. Having to teach it forced me to really challenge my own understanding and learn the detailed mechanics of what goes on, and that it is quite different from just “prompt tuning for every layer”

in prefix tuning, the model is still frozen, but instead of adding learnable tokens at the input, we add learned key and value vectors directly to each transformer layer. so for every layer, we create a small set of prefix K and prefix V vectors. during the forward pass, the real tokens' queries attend not only to the normal K and V from the sequence, but also to these learned prefix K and V. the prefix vectors themselves do not act like normal tokens and do not go through the feedforward layers; they exist only as extra K/V memory that the model can attend to.

because these prefixes are injected at every layer, they give the model much more expressive control than prompt tuning. instead of all influence having to come

from layer 0, prefix tuning can shape attention patterns and hidden states at each depth. all adaptation comes from these learned K/V vectors, while the main model weights stay frozen.

if this makes sense, i can continue with LORA next.

Student:

Good explanation — you've captured the core idea. But since we're doing this in a *Feynman-technique* style, I'm going to press you on **three subtle but important gaps** that distinguish a merely correct explanation from a *deep mechanistic* one.

Answer these, and your prefix-tuning understanding will be rock-solid.

? 1. Where do the prefix K/V vectors live in the computation graph?

You said:

"We add learned key and value vectors directly to each transformer layer."

But I need more precision:

- Are these prefix K/V concatenated to the existing K/V matrices?
- Do they participate in multi-head attention the same way as normal tokens?
- Do they get new Q vectors? (They shouldn't — but explain *why*.)
- How are the gradients routed so that only prefix K/V are updated?

Right now your description doesn't fully articulate *how* the prefix is injected mathematically.

? 2. You said "prefix vectors don't go through feedforward layers."

Then how do they influence the FFN output indirectly?

Since they never pass through the FFN, the only way prefix tuning can influence the FFN is by influencing:

- the attention output entering the residual stream.

Can you make explicit:

Which part of the residual stream is modified, and how does that propagate through subsequent FFNs?

Right now this chain is implied, but not explained.

? 3. Why is prefix tuning *much* more expressive than prompt tuning — mechanistically?

You gave the high-level idea ("injected at every layer"), but let's make it precise:

- What kinds of computations become possible when you can inject a fresh K/V set at layer L vs only at layer 0?
- Why does injecting new K/V at deeper layers allow the model to override or reshape internal feature transformations that prompt tuning cannot reach?
- How does this relate to depth-specific subspaces and the "feature hierarchy" inside transformers?

Try to articulate the *exact mechanism* by which prefix tuning "enters" the network at different depths — not just the intuition.

! One more subtle conceptual gap (bonus)

In prefix tuning, real tokens attend to prefix K/V.

But:

****Why do prefix K/V not need to correspond to any real token embedding?**

Why can they be "free-floating memory slots"? **

This is a major mechanistic distinction from prompt tuning, and you haven't described why this is possible.

After you answer:

Once you fill these gaps, **you may move on to LoRA.**



Here, the student also forced me to understand how gradient flows better

on your first question: the prefix K and V vectors are concatenated to the normal K and V matrices inside each layer's multi head attention. they participate exactly like extra memory slots: they are split across heads the same way as normal K and V, but they do not have corresponding queries. that's because prefix tuning never creates prefix hidden states, so there is no source for Q vectors; only the real tokens create Q. gradients flow only to the prefix parameters because the model weights themselves are frozen, so the only trainable tensors in the attention block are the prefix K and V tensors.

second point: even though prefixes don't go through the feedforward layers, they influence them indirectly through the residual stream. attention outputs combine information from real tokens and the prefix values, and that combined vector is added to the residual stream before the FFN. so the FFN receives a modified input, shaped by the prefix values the real tokens attended to, and that is how the influence propagates through all subsequent computations.

third point: prefix tuning is more expressive than prompt tuning because it lets you inject new information at every depth of the network. prompt tuning only perturbs the layer 0 hidden states, and the model must carry that influence upward through all layers, which limits what functions it can approximate. prefix tuning gives direct control over the internal representations at layer L, meaning the adapter can directly modulate mid-level or high-level features without relying on the early-layer residual stream. transformers learn depth-specific

subspaces, so having access to each layer's attention module gives prefix tuning a much larger adaptation space.

and on the last question: prefix K and V don't need to correspond to any token embedding because attention only requires key and value vectors, not full token states. the attention computation doesn't care whether a kv slot originated from a real sequence token or was directly parameterized. as long as the vectors are in the right space, the model can attend to them and use them as additional memory.

if that's clear, i can move on to lora next.

Student:

Perfect — that was a clean, fully mechanistic explanation.

You've now demonstrated expert-level understanding of prefix tuning.

Before you move on to LoRA, I want to highlight exactly what you improved (to reinforce the mental model), and then I'll invite you to begin the LoRA explanation.

What you now understand completely

1. Exact placement of prefix K/V

- They are *concatenated* to the layer's K/V matrices.
- They act as pure memory slots, not tokens.
- Only real tokens produce Q vectors.
- Gradients flow only into the prefix parameters because everything else is frozen.

This is textbook-correct.

2. How they influence the FFN indirectly

You cleanly articulated the residual-stream pathway:

prefix V → attention output → added to residual stream → modified FFN input.

This is the correct propagation mechanism (more important than people think).

3. Why prefix tuning is *much* more expressive than prompt tuning

Your answer nailed it:

- prefix tuning lets you inject information into *intermediate feature spaces*
- prompt tuning can only perturb layer-0 representations
- Transformers develop depth-specific subspaces, so deeper injections = more adaptation power

This is exactly how modern model editing perspectives describe these methods.

4. Why prefix K/V don't need to correspond to tokens

You correctly noted:

Attention requires only K/V, not full token embeddings.

This is a key insight distinguishing prefix tuning from any input-based method.

👍 **You're ready.**

Go ahead — **start explaining LoRA**, and I will again interrupt the moment anything is incomplete or imprecise.



in lora, the model weights are frozen, but we insert a low-rank update into certain weight matrices, usually the attention projection matrices like W_q , W_k , W_v , and W_o , and also the MLP projection layers (W_{up} , W_{down}). the idea is that instead of updating the full matrix W , we learn two small matrices A and B such that the effective weight becomes $W + BA$, where A is $(d \times r)$ and B is $(r \times d)$ with r

much smaller than d . during training, only A and B are updated; the original W never changes. at inference, the model just uses $W + BA$ as if it were the true weight.

because the update is low rank, lora adds far fewer parameters than full fine tuning, but it is much more expressive than prompt tuning or prefix tuning because it directly modifies the internal linear transformations of the network. instead of influencing the forward pass from the outside (like prompts or prefixes), lora adjusts the behavior of the attention or feedforward layers from inside the weight space. this lets lora adapt the model to more complex functions, while still avoiding catastrophic forgetting since the base weights stay frozen.

Student:

Great explanation — you're clearly very close to a full mechanistic understanding of LoRA.

As always, I'm going to push you with targeted questions that expose any remaining conceptual gaps, especially around *how* LoRA actually changes computations inside attention/MLP blocks.

Here are the key places where we need to refine or deepen the explanation.

? 1. You said the effective weight becomes $W + BA$.

But *where* in the computation graph does this addition happen?

For example, in an attention projection:

$$h_{\text{in}} W_q \longrightarrow Q$$

With LoRA, is it:

$$h_{\text{in}} (W_q + B_q A_q)$$

or

$$h_{\text{in}} W_q + (h_{\text{in}} B_q A_q)$$

Before this, I had a vague view of where LoRA is applied. Clarifying that the addition happens at the linear-projection output ($hW + hBA$) sharpened my understanding of how LoRA interacts with attention and FFN paths.

or something else?

These *look the same algebraically* but have different implementation implications (e.g., scaling, merging weights, memory layout, etc.).

You haven't yet stated **how the LoRA path is injected** in actual forward computation.

? 2. LoRA usually includes a scaling factor α/r .

What is the purpose of this scaling, mechanistically?

Is it:

Here, I realized I had omitted an essential stabilizing mechanism: α/r keeps the LoRA update small at initialization so it doesn't overwhelm the frozen model. This clarified why LoRA avoids destabilizing the residual stream.

- To stabilize training?
- To keep updates small so they don't distort the frozen residual stream?
- To ensure the model behaves like the base model early in training?

Right now you didn't mention this at all, but it's central to how LoRA avoids destabilizing attention projections.

? 3. Why is a *low-rank* update sufficient to represent complex task adaptations?

You said LoRA is "more expressive," but we should explore:

- Why does a rank- r matrix with tiny r allow surprisingly rich adaptation?
- How does this interact with the fact that transformers repeatedly apply linear projections and nonlinearities?
- Why doesn't the low rank bottleneck cripple expressive power?

This gets at the heart of LoRA's theoretical intuition.

? 4. You said LoRA "avoids catastrophic forgetting."

Mechanistically — *why*?

It's not enough to say "the base weights stay frozen."

Even if W is frozen:

- LoRA could still push BA to override the original behavior.
- So how does LoRA preserve original function unless the LoRA path is trained carefully?

I want a deeper causal explanation here.

? 5. What is the difference between updating W *directly* vs updating BA?

In other words:

- Why do low-rank adapters tend to be easier to optimize?
- How does the parameterization constrain the update space so the optimizer finds solutions efficiently?

This relates to optimization geometry, which your current explanation hasn't touched.

[This question from the student also made me learn that LoRA allows for a more efficient optimization \(beyond just compute cost and prevention of catastrophic forgetting\)](#)

? 6. Minimal mechanistic summary

After answering the above, I'll ask you to give a concise "mechanistic core" of LoRA (like you did for ICL/prompt/prefix tuning).

Answer these, and then we can declare your LoRA explanation complete.



on the first point:

the effective weight really is used as $W + BA$ inside the projection. in practice, the forward pass computes $h_{in} W + h_{in} (BA)$. some implementations (not pytorch or hf peft) sometimes fuse this into $h_{in} (W + BA)$ for efficiency, but conceptually it's just adding the low rank

path to the normal projection output. the addition happens at the output of the linear layer, before the attention or FFN logic continues.

on the scaling factor:

right, i should also clarify something i didn't mention earlier — the scaling factor α/r in lora. the low-rank update BA is usually multiplied by a factor α/r before being added to W . the purpose of this scaling is to keep the adapter's initial influence very small. at the start of training BA is basically near-zero, so without scaling it could distort the frozen model's projections too much and destabilize training. α sets the target magnitude of the update, and dividing by r normalizes it so that increasing the rank doesn't blow up the update size. this way the base model behaves almost exactly like W at initialization, and the adapter only takes over gradually as it learns something task specific.

on why a low rank update is expressive: even though BA is low rank, it gets applied at multiple layers and interacts with nonlinearities and depth. a small perturbation in the weight space, repeated through many layers, can produce rich functional changes. transformers also tend to have large, redundant subspaces, so many useful adaptations live in low-dimensional directions. in practice, a low-rank update is enough to shift the model into a task-specific subspace without needing a full-rank rewrite.

on catastrophic forgetting: lora avoids it because the pretrained W is untouched and always contributes to the output. the adapter can only add a direction in weight space; it cannot erase the pretrained function. the scaling also helps by keeping early updates small, so behavior stays close to the base model until training nudges the adapter into a task-relevant space. if the adapter is removed, the model instantly reverts to the original W .

The student's question prompted me to look deeper, and I learned that even a low-rank update can be powerful because it is applied inside deep nonlinear transformations. This updated my mental model: LoRA isn't expressive because BA is rich — but because BA perturbs many layers over depth.

on the difference between updating W directly vs updating BA : optimizing BA constrains the update to a small subspace, which makes training more stable. full fine tuning exposes millions of degrees of freedom and the optimizer can easily drift away from the pretrained solution. lora restricts the update to a low-dimensional manifold, which acts like a regularizer and makes optimization easier, since the search space is much smaller and better conditioned.

a minimal mechanistic summary: lora freezes the original weight matrix W and learns a low-rank matrix BA that is added to the projection output. BA is scaled so the adapter starts near zero and doesn't disrupt the base model. even with small rank, repeated perturbations through layers and nonlinearities give the method high expressive power. because W never changes, removing the adapter reverts the model instantly, which avoids forgetting and makes the method modular.

if this is clear, i can wrap up with a short comparison of icl, prompt tuning, prefix tuning and lora.

Student:

Excellent — this is a complete, mechanistically correct explanation of LoRA.

Your answers show you really understand *how* and *why* LoRA works, not just what it does.

Yes, you can now give the wrap-up comparison. Before you do, I'll highlight **what you explained extremely well**, to anchor the concepts:

What you now fully understand about LoRA

1. Correct computational injection point

You correctly stated that LoRA modifies the projection as:

$$h_{\text{in}}W + h_{\text{in}}(BA)$$

This is the right way to phrase it — LoRA perturbs the *output* of a linear projection, not the hidden state or residual directly.

2. Scaling factor (α/r) mechanics

Your explanation nailed the three purposes:

- keep the LoRA path initially near-zero
- ensure training stability
- prevent updates from exploding when rank increases

This is an essential detail many people forget.

3. Why low-rank is surprisingly expressive

You explained:

- deep networks amplify small perturbations
- nonlinearities + repetition → high expressive power
- transformer weight spaces contain many redundant / low-dimensional task-relevant directions

This is the exact intuition behind why $r \ll d$ still works.

4. Why LoRA avoids catastrophic forgetting

Perfect: LoRA *adds* directions; it never overwrites the base function.

You even clarified that removing the adapter cleanly reverts to W — that's the key property.

5. Optimization geometry

You correctly noted that BA constrains the update to a low-dimensional manifold, making training more stable and well-conditioned.

This is a deeper insight that only people who truly understand LoRA mention.

👍 **Go ahead.**

Please give your **short comparison** of:

- ICL
- Prompt tuning
- Prefix tuning
- LoRA

Try to make it:

- concise
- mechanistic
- emphasizing the differences in *where* adaptation enters the computation

I'll tell you if anything needs tightening, but given your performance so far, you'll likely produce a near-perfect summary.



to summarize: ICL adapts the model purely through the prompt by putting examples in the kv cache, so all "learning" happens inside the forward pass with no parameter updates. prompt tuning learns a small set of input-level soft prompt embeddings that steer the model from layer 0. prefix tuning is stronger because it injects learned key/value vectors at every layer, giving direct control over internal representations. lora is even more expressive because it adds low-rank updates inside the model's projection matrices while keeping the base weights frozen.

all four methods keep the pretrained model intact. in ICL, there is no trainable components; in the other 3, the trainable components are small and modular. that means you can train separate adapters for different tasks and swap them in without retraining the whole model, which

makes these approaches efficient and adaptable in practice.

Student:

Excellent summary — it is concise, correct, and mechanistically grounded.

Let me give you *one final refinement* to elevate it from "correct" to "expert-level," and then you're fully done:

Your summary (with slight sharpening)

Here is your summary with **one added sentence per method** that highlights the *location* of adaptation — the single most important conceptual axis.

ICL

- No weights change.
- All adaptation happens *inside the forward pass* by altering the **KV cache** with in-prompt examples.
- The model reinterprets the prompt structure using internal meta-learning heuristics learned during pretraining.

Prompt tuning

- Adds trainable **input-level embeddings**.
- Adaptation enters only through **layer-0 residual stream**, limiting depth of influence.
- Works best for shallow modifications to behavior.

Prefix tuning

- Injects learned **K/V vectors at every layer's attention module**.
- Adaptation enters at **all depths**, letting the prefixes manipulate intermediate feature spaces.
- Much more expressive because it can shape attention patterns throughout the network.

LoRA

- Adds **low-rank updates inside the weight matrices themselves** ($W \rightarrow W + BA$).

- Adaptation enters through the *linear transformations that generate Q/K/V and FFN activations*.
- Most expressive among modular/frozen-weight methods because it alters internal transformations rather than external signals.

Conceptual takeaway

The four methods differ entirely by **where** they inject adaptation:

Method	Adaptation Enters At	Strength
ICL	KV cache during forward pass	Weak–medium (no training required)
Prompt tuning	Layer-0 hidden states	Weak
Prefix tuning	K/V memory at every layer	Medium–strong
LoRA	Projection matrices inside each layer	Strong

This single factor explains almost all differences in expressive power.

If you'd like, you can now:

- make a diagram summarizing this,
- write a Feynman-style “teaching sheet,”
- or move on to another advanced topic (e.g., **P-tuning v2**, **adapter composition**, **why attention enables in-context learning**, etc.).

Just tell me what you want next.



Lets end of this session with you providing me a Feynman-style “teaching sheet”. It should include

generic prompt template for students to follow to learn any generic concept.

Below is a **Feynman-style Teaching Sheet** with a **generic prompt template** that students can use to learn *any* technical concept.

It follows the principles of:

- explaining simply,
- detecting gaps,
- stress-testing understanding,
- iterating toward clarity.



FEYNMAN-STYLE TEACHING SHEET

A reproducible framework for deeply understanding any concept

0. Setup

Use this sheet whenever you want to teach yourself a concept. The goal is to **force your brain to reconstruct the idea**, not merely recall it.

1. Define the Concept (Simple Words Only)

Explain the concept as if you are teaching a smart 12-year-old.

Template:

"The idea of ____ is that _____. It works by _____. The key reason we use it is _____."

Goal:

If you use jargon, you must define it.

If you cannot explain it simply → you don't yet understand it.

2. Explain the Mechanism (What Actually Happens)

Describe the *causal chain* or *step-by-step process* behind the concept.

Template:

"The process starts when ____ happens. Then ____ is computed. Finally, ____ produces the outcome.

The most important piece is ____ because it determines how ____ behaves."

Goal:

Capture the internals, not just the high-level intuition.

3. Give a Concrete Example

Pick an example that *forces* the concept to produce a clear output.

Template:

"For example, if we input ____, the system does ____, which leads to ____."

Goal:

If you cannot walk through a step-by-step example → your understanding is still abstract and fragile.

4. Identify What Makes It Work (The Core Insight)

Extract the **one or two principles** without which the concept wouldn't function.

Template:

"This only works because ____.

If we removed that, then ____ would fail."

Goal:

Reveal the structural backbone of the idea.

5. Contrast With Similar Concepts

Explain what it *is not*.

Comparison sharpens understanding.

Template:

*"This is different from ____ because ____.
While ____ works by ____, this works by ____."*

Goal:

Differentiate causes, mechanisms, or capabilities.

6. Stress-Test Your Understanding (Expose Gaps)

Ask yourself adversarial questions.

You can also ask ChatGPT to roleplay as a critical student (like we did).

Template:

*"If this is true, then why does ____ happen?"
"What would break if we changed ____?"
"What is the edge case that challenges my explanation?"*

Goal:

Gaps you discover here are the most valuable part of this method.

7. Repair the Gaps

Re-explain the confusing parts in simpler language.

Re-derive the mechanism until it "clicks."

Template:

*"The reason my explanation earlier was incomplete is ____."
"The actual mechanism is ____."*

8. Reassemble the Full Explanation (Simple + Correct)

Give the **final, clean explanation** after patching gaps.

Template:

*"Now that the gaps are fixed, the correct explanation is:
____."*

9. Teach It to Someone Else (OR simulate a student)

Teaching is the highest test of understanding.

Template:

"Let me teach you _____. Stop me whenever something is unclear or incomplete."

10. Optional: Create a One-Page Mental Model

This should include:

- Key mechanism
- Inputs / outputs
- Constraints
- Failure modes
- Why it exists

This becomes a permanent mental schema in your knowledge library.



GENERIC PROMPT TEMPLATE FOR STUDENTS

Use this whenever you want ChatGPT to help you learn a concept with the Feynman Technique.

Copy/Paste Prompt:

**I want to use the Feynman Technique to learn the concept: [insert concept].

Please act as a student who understands deep learning basics (or the field I'm in), and interrupt me whenever my explanation is unclear, incorrect, or incomplete. Your job is to ask sharp, probing questions that expose conceptual gaps.

I will begin by explaining the concept as if teaching a 12-year-old. After each block of my explanation, critique it and force me to refine it.

The last part - depending on your use case. For the above example, the AI should act as an undergrad who has some background of deep learning.